

Computer Vision:

Homework-7

Bilal Ahmed
ahmedb@purdue.edu

1 Theoretical Questions

Answer: Looking at different ways of extracting texture features, I think style transfer concept is simple yet powerful. I don't have an implementation but in my view we can couple the strategy of using channel normalization parameters with pyramidal representations used for key points detection algorithms like SIFT. Here is the step by step approach;

- Get pyramid of σ -smoothed images, that would capture information at varying levels of granularity.
- For each σ -smoothed image compute normalization parameters i.e. mean and std. deviation.
- Concatenate these normalization parameters (mean & std. deviation) obtained for each σ -smoothed image in all octaves.
- Use this concatenated descriptor as input to classifier like SVC.

2 Objective

The goal of this homework is to implement weather classifiers using different methods of extracting features. For training the classifier and evaluating it Multi-class weather dataset was used provided, having images of four categories namely cloudy, rain, sunrise and shine. Five different methods for extracting texture based features were used. For all the methods I used linear classifier from SKlearn.

Local Binary Pattern (LBP) Features.

LBP is method of extracting features using local texture information. Specifically, it compares each pixel value with the values in its neighborhood and creates a binary pattern based on these comparisons. These binary pattern are encoded efficiently by applying simple bitwise operations. Here is a step by step procedure;

- For each pixel (i,j), calculate values of pixels in the circle around it, with the positions offset of the neighbors on the circle given as;

$$(\Delta u, \Delta v) = \left(R \cos \left(\frac{2\pi p}{P} \right), R \sin \left(\frac{2\pi p}{P} \right) \right)$$

where $p = 0, 1, 2, \dots, P-1$. Bilinear interpolation is used to compute values of pixels not lying exactly on the grid.

- For each of these P neighbors, if their value is greater than the center pixel, bit value 1 is assigned otherwise 0, giving us a binary pattern.

- Binary pixel thus obtained is circularly shifted to get a run of leading zeros, such that the sequence should always end on 1 (if there is at least one 1 in the pattern, otherwise of course there will be all zeros). Such a binary pattern will correspond to minimum integer value corresponding to 1's and 0's in the pattern.
- The pattern thus obtained as encoding by assigning values as follows;
 - pattern of all zeros – encoded as 0,
 - pattern of all ones – encoded as P,
 - a run of zeros followed by a run of ones – encoded by integer number equal to the number of 1's in the run of 1's,
 - run of zeros followed by mix of 1's and 0's – encoded as P+1
- Then we make a histogram of these assigned codes in the entire image. This histogram is used as features for the classifier.

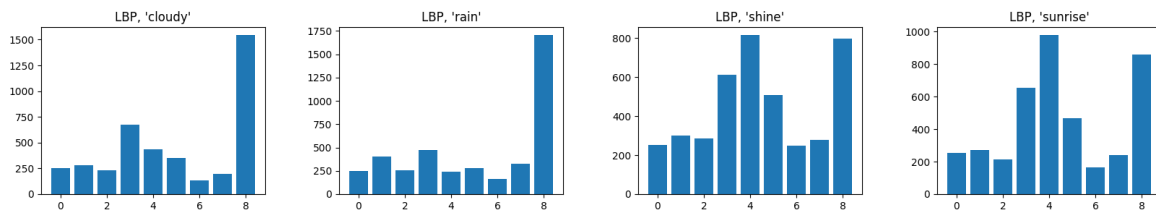


Figure 1: Example features using LBP

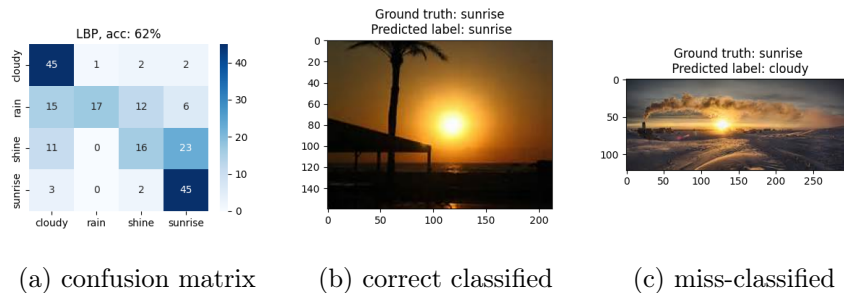


Figure 2: LBP: Confusion matrix, one example of correctly classified and mis-classified figures each.

We can get LBP features for any gray scale image, since we have RGB colored images, I converted RGB images to HSI and used hue channel to get LBP features. Fig.1 shows histogram features extracted using LBP for example images from each category. Fig.2 shows confusion matrix and one example of correctly classified image as well as one example of miss-classified image. The ground truth and predicted labels are mentioned in the titles of each image. I used $R = 1$ and $P = 8$ giving eight neighbors on the unit circle around each pixel.

Gram Matrix of Features from DNN.

Pre-trained Deep Neural Networks (DNN) trained to classify images extract useful features. We can use descriptors based on gram matrix of features from these DNNs. Specifically we were provided pre-trained VGG and RESNET and code to extract their features. Here is how their features were used to extract descriptors;

- Given F features from DNN having shape (N, M) where N is the number of channels and $M = W \times H$ obtained by flattening the remaining two dimensions.

- We get the gram matrix G ;

$$G = F \cdot (F)^\top,$$

where $G \in \mathbb{R}^{N \times N}$.

- Downsample G to size of 32×32 .
- Since G is symmetric, retain only upper triangle of is, including the diagonal, giving us descriptor of size $\mathbb{R}^{\frac{32(32+1)}{2}}$
- Descriptor thus obtained was used as input the linear SVM classifier.

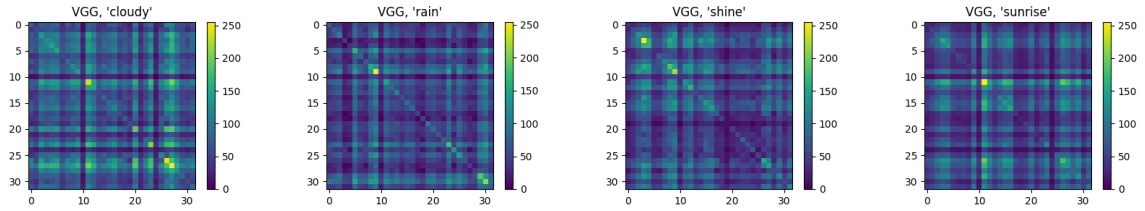


Figure 3: Example features using VGG

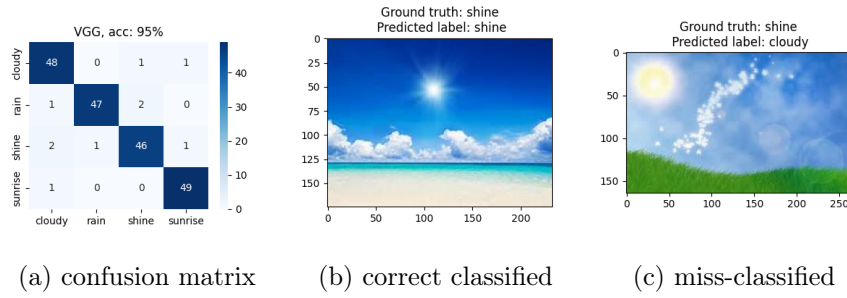


Figure 4: VGG: Confusion matrix, one example of correctly classified and mis-classified figures each.

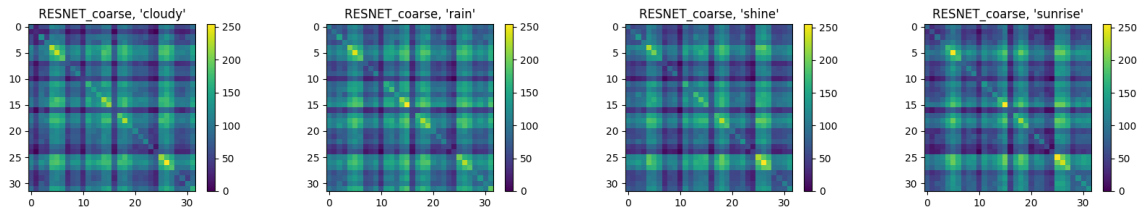


Figure 5: Example features using RESNET-coarse

Method of using gram matrix of features to get descriptors was applied to VGG and RESNET. For RESNET two types of descriptors were obtained using coarse and fine features from RESNET. Fig.3 shows the gram matrix for VGG features for all categories and Fig.4 shows summary of results (confusion matrix, correctly classified and miss-classified images) using VGG.

Fig.5 shows the gram matrix for RESNET (coarse) features for all categories and Fig.6 shows summary of results (confusion matrix, correctly classified and miss-classified images) using RESNET (coarse).

Fig.7 shows the gram matrix for RESNET (fine) features for all categories and Fig.6 shows summary of results (confusion matrix, correctly classified and miss-classified images) using RESNET (fine).

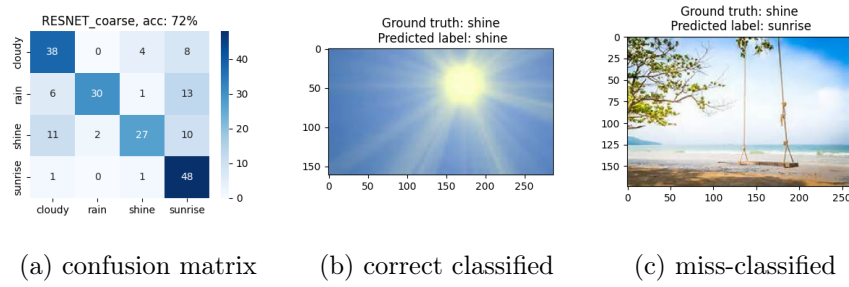


Figure 6: RESNET-coarse: Confusion matrix, one example of correctly classified and mis-classified figures each.

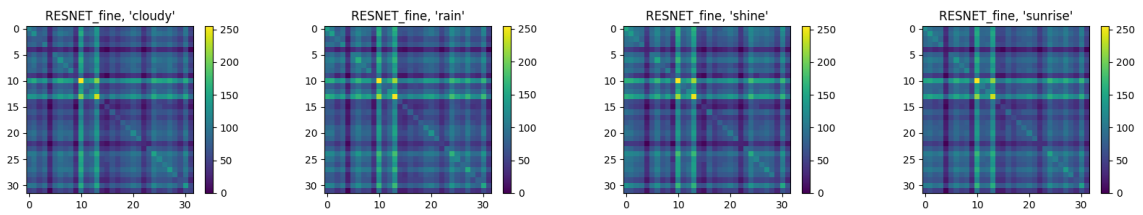


Figure 7: Example features using RESNET-fine

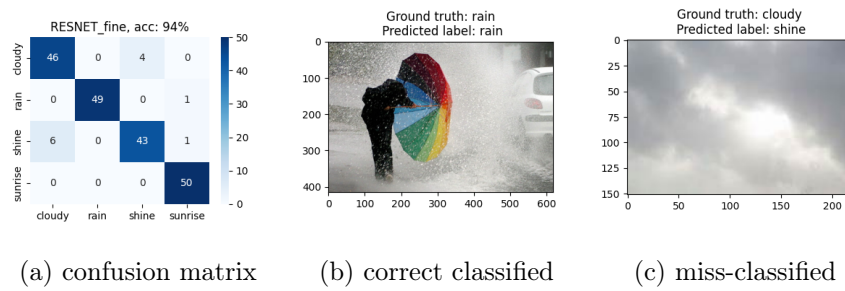


Figure 8: RESNET-fine: Confusion matrix, one example of correctly classified and mis-classified figures each.

Extra Credits (AdaIN).

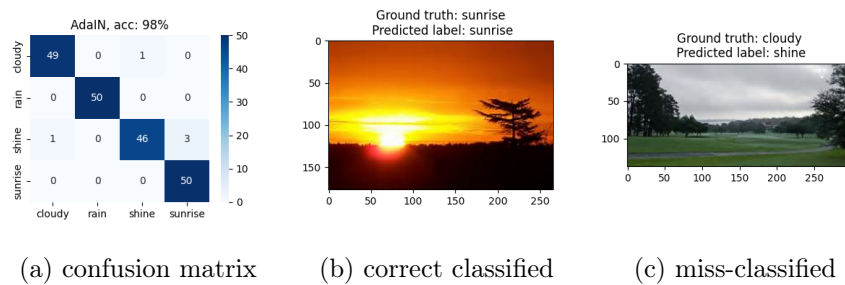


Figure 9: AdaIN: Confusion matrix, one example of correctly classified and mis-classified figures each.

Another way of extracting the descriptors is using the concept of style transfer. The idea is that the information about the texture or style is captured by the channel normalization parameters i.e. per channel mean and std. deviation. So we use the channel means and std. deviations as descriptors. So using the feature maps from VGG, instead of computing the gram matrix, I calculated their per channel means and std. deviations and concatenated them together to get a long descriptor. Since there are 512 channels, getting descriptor as explain above gives 1024 features. Using these descriptors, I was able to beat all the previous results, with accuracy increasing to 98%. Fig.9 shows confusion matrix, correctly classified example and miss-classified example using AdaIN method.

Discussion.

Table.1 shows the summary of accuracies obtained using different method of extracting descriptors. As we can see the handcrafted method of extracting style information about the image used to struggle with extracting accurate information. But the neural network based classifiers are able to learn richer features on their own. Applying the same classifier to the NN based features gives way better results than the conventional methods like LBP. VGG and RESNET (fine) give very similar results 95% and 94% respectively. Whereas RESNET (coarse) is not that impressive, which makes sense since it is not intended to capture the fine details.

Even more impressive results are obtained using adaptive instance norm (AdaIN). This uses concept of style transfer that is based on the idea that style information is captured by channel normalization parameters. So if just use these per channel summary statistics, we can get even more accurate results, as depicted by an accuracy of 98%.

Table 1: Classification accuracy using different methods of features extraction.

Method	Accuracy
LBP	62 (%)
VGG	95 (%)
RESNET (coarse)	72 (%)
RESNET (fine)	94 (%)
AdaIN	98 (%)

Python Code

```

1 import cv2
2 import pathlib
3
4 import scipy
5 import computer_vision
6 import BitVector
7
8 import matplotlib.pyplot as plt
9 # import matplotlib as mpl
10 import numpy as np
11 # import math
12
13 import os
14 import glob
15 import time
16 import torch
17
18 from sklearn.svm import SVC, LinearSVC

```

```

19 from sklearn.metrics import accuracy_score, confusion_matrix
20 import seaborn as sns
21 torch.set_grad_enabled(False)
22
23 from vgg_and_resnet import VGG19, CustomResNet
24
25
26 class FeatureExtractor:
27     """Implements functionality to extract texture based features from the
28     image and save them to the path specified.
29     """
30     def __init__(self, categories, data_dir, sub_dir='data') -> None:
31         self.categories = np.array(categories)
32         self.data_dir = data_dir
33         self.sub_dir = sub_dir
34
35         self.output_dir = os.path.join(self.data_dir, 'features')
36         # Create the directory (and any necessary parent directories)
37         os.makedirs(self.output_dir, exist_ok=True)
38
39     def get_train_test_splits(self, method='LBP'):
40         """Returns data as train-test splits"""
41         train_features, test_features = self.get_features(method=method)
42         train_x, train_y, train_image_names = FeatureExtractor.
43             get_inputs_and_labels(train_features, self.categories)
44
45         test_x, test_y, test_image_names = FeatureExtractor.
46             get_inputs_and_labels(test_features, self.categories)
47         return train_x, train_y, test_x, test_y, test_features,
48             test_image_names
49
50     def get_features(self, method='LBP'):
51         """Returns features for the method specified.
52
53         Args:
54             method: choices = ['LBP', 'VGG', 'RESNET_coarse', 'RESNET_fine',
55                               'AdaIN']
56         """
57         choices = ['LBP', 'VGG', 'RESNET_coarse', 'RESNET_fine', 'AdaIN']
58         assert method in choices, print(f"Select method from {choices}...")
59         print(f"Reading features for '{method}' from disk...")
60         filepath = os.path.join(self.output_dir, f'{method}_training.npz')
61         train_features = np.load(filepath, allow_pickle=True)['features'].
62             item()
63
64         filepath = os.path.join(self.output_dir, f'{method}_testing.npz')
65         test_features = np.load(filepath, allow_pickle=True)['features'].
66             item()
67
68         return train_features, test_features
69
70     def extract_and_save_AdaIN_features(self):
71         """Extracts AdaIN features and saves them to the disk."""
72         print(f"For AdaIN features using VGG features...")
73         vgg = VGG19()
74         vgg.load_weights('vgg_normalized.pth')
75         splits = ['training', 'testing']
76         for split in splits:
77             print(f"For '{split}'...")

```

```

72     images_dir = os.path.join(self.data_dir, self.sub_dir, split)
73
74     filepaths = glob.glob(os.path.join(images_dir, "*.jpg"))
75     AdaIN_features = {}
76
77     for filepath in filepaths:
78
79         try:
80             img = FeatureExtractor.read_image_from_path(filepath)
81             AdaIN_feat = FeatureExtractor.
82                 get_AdaIN_features_for_image(img, vgg)
83
84             image_name = os.path.basename(filepath)
85             image_name = image_name.rsplit('.', 1)[0]
86             AdaIN_features[image_name] = AdaIN_feat
87         except:
88             print(f"Error loading image: {os.path.basename(filepath)}")
89
90     out_path = os.path.join(self.output_dir, f'AdaIN_{split}')
91     np.savez_compressed(out_path, features=AdaIN_features)
92     print(f"AdaIN features saved to {out_path}.npz")
93
94 def get_AdaIN_features_for_image(img, vgg):
95     """Extracts VGG features for the given image.
96     """
97     x = cv2.resize(img, (256, 256), interpolation=cv2.INTER_AREA)
98     # Obtain the output feature map
99     vgg_feature = vgg(x)
100     vgg_feature = vgg_feature.reshape(vgg_feature.shape[0], -1)
101     mu = np.mean(vgg_feature, axis=1)
102     sigma = np.std(vgg_feature, axis=1)
103
104     return np.concatenate([mu, sigma])
105
106
107 def extract_and_save_RESNET50_features(self):
108     """Extracts LBP features and saves them to the disk."""
109     print(f"Using RESNET-50 to extract features...")
110     encoder_name='resnet50'
111     resnet = CustomResNet(encoder=encoder_name)
112     splits = ['training', 'testing']
113     for split in splits:
114         print(f"For '{split}'...")
115         images_dir = os.path.join(self.data_dir, self.sub_dir, split)
116
117         filepaths = glob.glob(os.path.join(images_dir, "*.jpg"))
118         RESNET_coarse_features = {}
119         RESNET_fine_features = {}
120
121         for filepath in filepaths:
122
123             try:
124                 img = FeatureExtractor.read_image_from_path(filepath)
125                 resnet_coarse, resnet_fine = FeatureExtractor.
126                     get_RESNET50_features_for_image(img, resnet)
127
128                 image_name = os.path.basename(filepath)

```

```

128         image_name = image_name.rsplit('.', 1)[0]
129         RESNET_coarse_features[image_name] = resnet_coarse
130         RESNET_fine_features[image_name] = resnet_fine
131     except:
132         print(f"Error loading image: {os.path.basename(filepath)}")
133
134     # saving coarse features
135     out_path = os.path.join(self.output_dir, f'RESNET_coarse_{split}')
136     np.savez_compressed(out_path, features=RESNET_coarse_features)
137     print(f"RESNET_coarse features saved to {out_path}.npz")
138
139     # saving fine features
140     out_path = os.path.join(self.output_dir, f'RESNET_fine_{split}')
141     np.savez_compressed(out_path, features=RESNET_fine_features)
142     print(f"RESNET_fine features saved to {out_path}.npz")
143
144 def get_RESNET50_features_for_image(img, resnet):
145     """Extracts RESNET50 features for the given image.
146     """
147     x = cv2.resize(img, (256, 256), interpolation=cv2.INTER_AREA)
148     # Obtain the output feature map
149
150     resnet_feat_coarse, resnet_feat_fine = resnet(x)
151
152     resnet_feat_coarse = resnet_feat_coarse.reshape(resnet_feat_coarse.
153                                                     shape[0], -1)
154     gram_mat_coarse = np.matmul(resnet_feat_coarse, resnet_feat_coarse.T)
155     gram_mat_coarse = cv2.resize(gram_mat_coarse, (32, 32),
156                                     interpolation=cv2.INTER_AREA)
157
158     resnet_feat_fine = resnet_feat_fine.reshape(resnet_feat_fine.shape
159                                                 [0], -1)
160     gram_mat_fine = np.matmul(resnet_feat_fine, resnet_feat_fine.T)
161     gram_mat_fine = cv2.resize(gram_mat_fine, (32, 32), interpolation=
162                                 cv2.INTER_AREA)
163     return gram_mat_coarse, gram_mat_fine
164
165 def extract_and_save_VGG_features(self):
166     """Extracts VGG features and saves them to the disk."""
167     print(f"Using VGG to extract features...")
168     vgg = VGG19()
169     vgg.load_weights('vgg_normalized.pth')
170     splits = ['training', 'testing']
171     for split in splits:
172         print(f"For '{split}'...")
173         images_dir = os.path.join(self.data_dir, self.sub_dir, split)
174
175         filepaths = glob.glob(os.path.join(images_dir, "*.jpg"))
176         VGG_features = {}
177
178         for filepath in filepaths:
179             try:
180                 img = FeatureExtractor.read_image_from_path(filepath)
181                 VGG_feature = FeatureExtractor.
182                     get_VGG_features_for_image(img, vgg)

```

```

179         image_name = os.path.basename(filepath)
180         image_name = image_name.rsplit('.', 1)[0]
181         VGG_features[image_name] = VGG_feature
182     except:
183         print(f"Error loading image: {os.path.basename(filepath)}")
184
185     out_path = os.path.join(self.output_dir, f'VGG_{split}')
186     np.savez_compressed(out_path, features=VGG_features)
187     print(f"VGG features saved to {out_path}.npz")
188
189 def get_VGG_features_for_image(img, vgg):
190     """Extracts VGG features for the given image.
191     """
192     x = cv2.resize(img, (256, 256), interpolation=cv2.INTER_AREA)
193     # Obtain the output feature map
194     vgg_feature = vgg(x)
195     vgg_feature = vgg_feature.reshape(vgg_feature.shape[0], -1)
196     gram_mat = np.matmul(vgg_feature, vgg_feature.T)
197     gram_mat = cv2.resize(gram_mat, (32, 32), interpolation=cv2.
198         INTER_AREA)
199     return gram_mat
200
201 def extract_and_save_LBP_features(self, R=1, P=8, resize=True):
202     """Extracts LBP features and saves them to the disk."""
203     splits = ['training', 'testing']
204     for split in splits:
205         print(f"For '{split}'...")
206         images_dir = os.path.join(self.data_dir, self.sub_dir, split)
207
208         filepaths = glob.glob(os.path.join(images_dir, "*.jpg"))
209         LBP_features = {}
210
211         for filepath in filepaths:
212             try:
213                 img = FeatureExtractor.read_image_from_path(filepath)
214                 LBP = FeatureExtractor.get_LBP_features_for_image(img, R
215                     =R, P=P, resize=resize)
216
217                 image_name = os.path.basename(filepath)
218                 image_name = image_name.rsplit('.', 1)[0]
219                 LBP_features[image_name] = LBP
220             except:
221                 print(f"Error loading image: {os.path.basename(filepath)}")
222
223         out_path = os.path.join(self.output_dir, f'LBP_{split}')
224         np.savez_compressed(out_path, features=LBP_features)
225         print(f"LBP features saved to {out_path}.npz")
226
227 def get_LBP_features_for_image(img, R=1, P=8, resize=True):
228     """Extracts 'local binary pattern (LBP)' features
229     for the given image.
230     """
231     if resize:
232         # downsample image to 64x64
233

```

```

234         new_dimensions = (64, 64)
235         downsampled_image = cv2.resize(img, new_dimensions,
236                                         interpolation=cv2.INTER_AREA)
237         HSI_img = FeatureExtractor.RGB_to_HSI(downsampled_image)
238     else:
239         HSI_img = FeatureExtractor.RGB_to_HSI(img)
240     H = HSI_img[...,0]
241     ### Local binary pattern
242     LBP_features = np.zeros_like(H)
243
244     neighbor_offsets = np.array([(R*np.cos(2*np.pi*p/P), R*np.sin(2*np.
245                                pi*p/P)) for p in range(P)])
246     pixel_y, pixel_x = np.meshgrid(range(1, H.shape[0]-1), range(1, H.
247                                shape[1]-1))
248     pixels = np.stack([pixel_x.flatten(), pixel_y.flatten()])
249
250     neighboring_pixels = pixels[None,:,:] + neighbor_offsets[:, :, None]
251
252     central_values, _ = FeatureExtractor.bilinear_interpolation(H,
253                        pixels)
254     neighboring_values = np.zeros((P, neighboring_pixels.shape[-1]))
255     for i in range(P):
256         img_values, _ = FeatureExtractor.bilinear_interpolation(H,
257                        neighboring_pixels[i])
258         neighboring_values[i] = img_values.squeeze()
259
260     binary_patterns = np.zeros(neighboring_values.shape, dtype=np.bool)
261     binary_patterns[neighboring_values > central_values] = 1
262
263     encodings = FeatureExtractor.get_LBP_encodings(binary_patterns)
264
265     LBP_features[pixel_y.flatten(), pixel_x.flatten()] = encodings
266
267     return np.histogram(LBP_features.flatten(), bins=np.arange(P+2))[0]
268
269
270 @staticmethod
271 def get_LBP_encodings(binary_patterns):
272     """Returns encodings for the input binary patterns.
273     Args:
274         binary_patterns: (P, N)
275
276     Returns:
277         np array of size N
278     """
279     P, N = binary_patterns.shape
280     encodings = []
281     for n in range(N):
282         bv = BitVector.BitVector(bitlist=binary_patterns[:,n])
283
284         minbv = BitVector.BitVector(intVal = min([int(bv<<1) for p in
285                                range(P)]), size=P)
286         runs = minbv.runs()
287         if len(runs) == 1 and runs[0] == 0:
288             enc = 0
289         if len(runs) == 1 and runs[0] != 0:
290             enc = P
291         elif len(runs) == 2:
292             enc = len(runs[1])

```

```

287         else:
288             enc = P+1
289
290         encodings.append(enc)
291     return np.array(encodings)
292
293 @staticmethod
294 def RGB_to_HSI(RGB_img):
295     """Converts RGB for the input image, to HSI"""
296     RGB_img = RGB_img/255.0
297     R = RGB_img[...,0]
298     G = RGB_img[...,1]
299     B = RGB_img[...,2]
300
301     H = np.zeros(RGB_img.shape[:2])
302     S = np.zeros(RGB_img.shape[:2])
303     M = np.max(RGB_img, axis=2)
304     m = np.min(RGB_img, axis=2)
305     c = M - m
306
307     mask = (M == R) & (c != 0)
308     H[mask] = 60*((G[mask] - B[mask])/c[mask]) % 6)
309
310
311     mask = (M == G) & (c != 0)
312     H[mask] = 60*((B[mask] - R[mask])/c[mask]) + 2)
313
314     mask = (M == B) & (c != 0)
315     H[mask] = 60*((R[mask] - G[mask])/c[mask]) + 4)
316
317     I = (R + G + B)/3
318
319     mask = c != 0
320     S[mask] = 1 - m[mask]/c[mask]
321
322     return np.stack([H,S,I], axis=-1)
323
324 @staticmethod
325 def read_image_from_path(image_path):
326     """Reads images from the subdir"""
327     img_bgr = cv2.imread(image_path)
328     # Convert the image from BGR to RGB format
329     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
330     return img_rgb
331
332 @staticmethod
333 def bilinear_interpolation(img, coordinates):
334     """For the given img and coordinates, uses bilinear interpolation
335     and returns img values for the input coordinates.
336     Args:
337         transformed_coordinates = (2, n)
338     """
339     # making sure img is 3 dimensional....
340     if img.ndim == 2:
341         img = img[...,None]
342     img_height, img_width = img.shape[:2]
343     # Extract integer and fractional parts of coordinates
344     x_trans = coordinates[0]
345     y_trans = coordinates[1]

```

```

346     x_floor = np.floor(x_trans).astype(int)
347     y_floor = np.floor(y_trans).astype(int)
348     x_ceil = np.ceil(x_trans).astype(int)
349     y_ceil = np.ceil(y_trans).astype(int)
350
351     # Clip coordinates to image boundaries
352     x_floor = np.clip(x_floor, 0, img_width - 1)
353     y_floor = np.clip(y_floor, 0, img_height - 1)
354     x_ceil = np.clip(x_ceil, 0, img_width - 1)
355     y_ceil = np.clip(y_ceil, 0, img_height - 1)
356
357     # Bilinear interpolation weights
358     wx = x_trans - x_floor # fractional part in x
359     wy = y_trans - y_floor # fractional part in y
360
361     # Perform bilinear interpolation
362     img_values = ((1 - wx) * (1 - wy))[:, None] * img[y_floor, x_floor]
363     + \
364         (wx * (1 - wy))[:, None] * img[y_floor, x_ceil] + \
365         ((1 - wx) * wy)[:, None] * img[y_ceil, x_floor] + \
366         (wx * wy)[:, None] * img[y_ceil, x_ceil]
367     # Create a mask for valid (within image bounds) coordinates
368     mask_coordinates = (x_trans >= 0) & (x_trans < img_width) & \
369         (y_trans >= 0) & (y_trans < img_height)
370     return img_values.squeeze(), mask_coordinates.squeeze()
371
372 @staticmethod
373 def get_inputs_and_labels(features, categories):
374     x = []
375     y = []
376     image_names = {}
377     for i, (image_name, feat) in enumerate(features.items()):
378         if feat.ndim == 2:
379             # extract upper trianlge elements...
380             n,m =feat.shape
381             feat = feat[np.triu_indices(n, m=m)]
382             x.append(feat)
383             y.append(next((i for i, label in enumerate(categories) if label
384                 in image_name), -1))
385
386             image_names[i] = image_name
387     x = np.array(x)
388     y = np.array(y)
389     return x, y, image_names
390
391 class Classifier:
392     def __init__(self, method, categories, output_dir, data_dir, subdir='
393     data') -> None:
394         """Creates an instance of feature extractor and uses that to get
395         train/test splits of data. In addition, it implements linear fit
396         and provides plotting functions.
397         """
398         choices = ['LBP', 'VGG', 'RESNET_coarse', 'RESNET_fine', 'AdaIN']
399         assert method in choices, print(f"Select method from {choices}!")
400
401         feature_extractor = FeatureExtractor(categories, data_dir, sub_dir=
402         subdir)

```

```

401     train_x, train_y, test_x, test_y, test_features, test_image_names =
402         feature_extractor.get_train_test_splits(method=method)
403     self.test_x, self.test_y, self.test_features = test_x, test_y,
404         test_features
405     self.test_image_names = test_image_names
406
407     self.svm_classifier = LinearSVC()
408     print(f"Fitting classifier for {method}...")
409     self.svm_classifier.fit(train_x, train_y)
410
411     self.categories = categories
412     self.method = method
413     self.output_dir = output_dir
414     self.data_dir = data_dir
415     self.subdir = subdir
416
417 def plot_confusion_matrix(self, savefig=True):
418     """Using the trained linear classifier, and test data,
419     plots confusion matrix as well as computes accuracy.
420     """
421     pred_y = self.svm_classifier.predict(self.test_x)
422
423     # Evaluate the classifier
424     acc = accuracy_score(self.test_y, pred_y)
425     conf_matrix = confusion_matrix(self.test_y, pred_y)
426     fig, ax = plt.subplots(figsize=(4,3))
427     sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
428                 xticklabels=categories, yticklabels=categories, ax=ax)
429
430     ax.set_title(f"{self.method}, acc: {acc*100:.0f}%")
431     if savefig:
432         filepath = os.path.join(self.output_dir, f'{self.method}
433             _c_matrix.png')
434         plt.savefig(filepath)
435         print(f"For {self.method}: \nconfusion matrix saved to: {
436             filepath}")
437
438 def plot_example_features(self):
439     """plots randomly choosen example features for all categories"""
440     figsize = (4,3)
441     filenames = np.array(list(self.test_features.keys()))
442     print(f"For {self.method}: ")
443     for cat in self.categories:
444         filename = np.random.choice(filenames[np.char.find(filenames,
445             cat) != -1])
446         features = self.test_features[str(filename)]
447         fig, ax = plt.subplots(figsize=figsize)
448         if self.method == 'LBP':
449             ax.bar(np.arange(features.shape[0]), features)
450         else:
451             features = 255*(features - np.min(features))/(np.max(
452                 features) - np.min(features))
453             mappable = ax.imshow(features.astype(np.uint8), cmap='
454                 viridis')
455             plt.colorbar(mappable)
456         ax.set_title(f"{self.method}, '{cat}'")
457         filepath = os.path.join(self.output_dir, f'{self.method}
458             _features_{cat}.png')

```

```

452         plt.savefig(filepath)
453         print(f"\t {cat} example features saved to: {filepath}")
454
455     def plot_qualitative_results(self, figsize=(4,3)):
456         """plots one correctly classified and one mis-classified images"""
457
458         pred_y = self.svm_classifier.predict(self.test_x)
459
460         # correct example
461         ind_correct = int(np.random.choice(np.where(pred_y == self.test_y)
462         [0]))
463         filename = self.test_image_names[ind_correct]
464         ground_truth_label = self.categories[self.test_y[ind_correct]]
465         predicted_label = self.categories[pred_y[ind_correct]]
466         self.display_image(filename, ground_truth_label, predicted_label,
467         correct=True, figsize=figsize)
468
469         # incorrect example
470         ind_correct = int(np.random.choice(np.where(pred_y != self.test_y)
471         [0]))
472         filename = self.test_image_names[ind_correct]
473         ground_truth_label = self.categories[self.test_y[ind_correct]]
474         predicted_label = self.categories[pred_y[ind_correct]]
475         self.display_image(filename, ground_truth_label, predicted_label,
476         correct=False, figsize=figsize)
477
478     def display_image(self, filename, ground_truth, predicted_label, correct
479     =True, figsize=(4,3)):
480         """Displays image with the given filename, adds ground truth
481         and predicted labels.
482         """
483         filepath = os.path.join(data_dir, subdir, 'testing', f"{filename}.
484         jpg")
485         img = FeatureExtractor.read_image_from_path(filepath)
486         fig, ax = plt.subplots(figsize=(4,4))
487         ax.imshow(img)
488         ax.set_title(f"Ground truth: {ground_truth}\n Predicted label: {
489         predicted_label}")
490         if correct:
491             ind = 'correct'
492         else:
493             ind = 'missclassified'
494         filepath = os.path.join(self.output_dir, f"{self.method}_example_{
495         ind}.png")
496         plt.savefig(filepath)
497         print(f"\t example of {ind} result saved to: {filepath}")
498
499     if __name__ == '__main__':
500
501         categories = ['cloudy', 'rain', 'shine', 'sunrise']
502         output_dir = r'C:\Users\ahmedb\projects\computer-vision\images\hw7'
503         data_dir = r'C:\Users\ahmedb\projects\computer-vision\HW7-Auxilliary'
504         subdir = 'data'
505
506         # loading features and saving to disk.
507         feature_extractor = FeatureExtractor(categories, data_dir, sub_dir=
508         subdir)
509         feature_extractor.extract_and_save_AdaIN_features()

```

```
502 feature_extractor.extract_and_save_LBP_features()
503 feature_extractor.extract_and_save_VGG_features()
504 feature_extractor.extract_and_save_RESNET50_features()
505
506 # using the saved features, implements classifier and generates results
507 methods = ['LBP', 'VGG', 'RESNET_coarse', 'RESNET_fine', 'AdaIN']
508 for method in methods:
509     classifier = Classifier(method, categories, output_dir, data_dir,
510                             subdir)
511
512     if method != 'AdaIN':
513         # plot example features
514         classifier.plot_example_features()
515
516     # confusion matrix
517     classifier.plot_confusion_matrix()
518
519     # plot qualitative results
520     classifier.plot_qualitative_results(figsize=(4,3))
```

Listing 1: Example Python code